

A Laboratory Manual for

Machine Learning

(3154602)

B.E. Semester 5

Computer Science & Engineering

(Data Science)

Institute logo



Directorate of Technical Education, Gandhinagar,

Gujarat

**Vishwakarma Government Engineering College,
Chandkheda**

Certificate

This is to certify that Mr./Ms. Bhavik Rohi Enrollment No. 220170146061 of B.E. Semester 5 Computer Science & Engineering (Data Science) of this Institute (GTU Code: 046) has satisfactorily completed the Practical / Tutorial work for the subject **Machine Learning (3154602)** for the academic year 2024-2025.

Place: _____

Date: _____

Name and Sign of Faculty member

Head of the Department

Preface

Main motto of any laboratory/practical/field work is for enhancing required skills as well as creating ability amongst students to solve real time problem by developing relevant competencies in psychomotor domain. By keeping in view, GTU has designed competency focused outcome-based curriculum for engineering degree programs where sufficient weightage is given to practical work. It shows importance of enhancement of skills amongst the students and it pays attention to utilize every second of time allotted for practical amongst students, instructors and faculty members to achieve relevant outcomes by performing the experiments rather than having merely study type experiments. It is must for effective implementation of competency focused outcome-based curriculum that every practical is keenly designed to serve as a tool to develop and enhance relevant competency required by the various industry among every student. These psychomotor skills are very difficult to develop through traditional chalk and board content delivery method in the classroom. Accordingly, this lab manual is designed to focus on the industry defined relevant outcomes, rather than old practice of conducting practical to prove concept and theory.

By using this lab manual students can go through the relevant theory and procedure in advance before the actual performance which creates an interest and students can have basic idea prior to performance. This in turn enhances pre-determined outcomes amongst students. Each experiment in this manual begins with competency, industry relevant skills, course outcomes as well as practical outcomes (objectives). The students will also achieve safety and necessary precautions to be taken while performing practical.

This manual also provides guidelines to faculty members to facilitate student centric lab activities through each experiment by arranging and managing necessary resources in order that the students follow the procedures with required safety and necessary precautions to achieve the outcomes. It also gives an idea that how students will be assessed by providing rubrics.

Machine learning is one of the prominent field of research. Prediction is very essential task in almost every business. This is the era of data. Finding useful patterns and making prediction greatly affect the success of any business. Machine learning algorithms provide opportunities to do statistical analysis of data, which in turn helps to take business decisions.

Utmost care has been taken while preparing this lab manual however always there is chances of improvement. Therefore, we welcome constructive suggestions for improvement and removal of errors if any.

Practical – Course Outcome matrix

Course Outcomes (COs): CO-1: Demonstrate fundamentals of machine learning. CO-2: Understand the importance of feature selection and dimension reduction techniques to deal with curse of dimensionality. CO-3: Apply machine learning algorithms to solve various problems. CO-4: Devise and develop a machine learning model using various steps. CO-5: Evaluate the performance of machine learning models.						
Sr. No.	Objective(s) of Experiment	CO1	CO2	CO3	CO4	CO5
1.	Find statistical measures such as Mean, Median , Mode , Standard Deviation and Variance of the given data	√				
2.	Implement data dispersion measure Five Number Summary, detect an outliers and generate box using python libraries	√	√			
3.	Implement program to perform dimension reduction of the high dimension data	√	√			
4.	Implement Linear Regression and Logistic Regression models	√		√		
5.	Implement Emotion detection system of a person using naïve Bayes classifier and evaluate model performance			√	√	√
6.	Implement k-NN, SVM, Decision tree classifiers on dataset and test its performance with comparision			√	√	√
7.	Implement program to demonstrate Neural Network Classifier and tune hyper parameters.			√	√	√
8.	Write a program to demonstrate within class scatter, between class scatter and total scatter of the dataset			√	√	√
9.	Write a program to demonstrate clustering using K-means algorithm			√	√	√
10.	Solve Real world problem with appropriate dataset using Machine Learning Model in Python Programming Language			√	√	√

Industry Relevant Skills

The following industry relevant competency are expected to be developed in the student by undertaking the practical work of this laboratory.

1. Apply knowledge of Machine Learning to solve real world problems
2. Understand and analyze the data analytically to use them wisely to build the ML model

Guidelines for Faculty members

1. Teacher should provide the guideline with demonstration of practical to the students with all features.
2. Teacher shall explain basic concepts/theory related to the experiment to the students before starting of each practical
3. Involve all the students in performance of each experiment.
4. Teacher is expected to share the skills and competencies to be developed in the students and ensure that the respective skills and competencies are developed in the students after the completion of the experimentation.
5. Teachers should give opportunity to students for hands-on experience after the demonstration.
6. Teacher may provide additional knowledge and skills to the students even though not covered in the manual but are expected from the students by concerned industry.
7. Give practical assignment and assess the performance of students based on task assigned to check whether it is as per the instructions or not.
8. Teacher is expected to refer complete curriculum of the course and follow the guidelines for implementation.

Instructions for Students

1. Students are expected to carefully listen to all the theory classes delivered by the faculty members and understand the COs, content of the course, teaching and examination scheme, skill set to be developed etc.
2. Students shall organize the work in the group and make record of all observations.
3. Students shall develop maintenance skill as expected by industries.
4. Student shall attempt to develop related hand-on skills and build confidence.
5. Student shall develop the habits of evolving more ideas, innovations, skills etc. apart from those included in scope of manual.
6. Student shall refer technical magazines and data books.
7. Student should develop a habit of submitting the experimentation work as per the schedule and s/he should be well prepared for the same.

Index
(Progressive Assessment Sheet)

Sr. No.	Objective(s) of Experiment	Page No.	Date of performance	Date of submission	Assessment Marks	Sign. of Teacher with date	Remarks
1	Find statistical measures such as Mean, Median , Mode , Standard Deviation and Variance of the given data						
2	Implement data dispersion measure Five Number Summary, detect an outliers and generate box using python libraries						
3	Implement program to perform dimension reduction of the high dimension data						
4	Implement Linear Regression and Logistic Regression models						
5	Implement Emotion detection system of a person using naïve Bayes classifier and evaluate model performance						
6	Implement k-NN, SVM, Decision tree classifiers on dataset and test its performance with comparison						
7	Implement program to demonstrate Neural Network Classifier and tune hyper parameters.						
8	Write a program to demonstrate within class scatter, between class scatter and total scatter of the dataset						
9	Write a program to demonstrate clustering using K-means algorithm						
10	Solve Real world problem with appropriate dataset using Machine Learning Model in Python Programming Language						
Total							

Experiment No: 1

Find statistical measures such as Mean, Median and Mode, Standard Deviation and Variance of the given data

Date:

Competency and Practical Skills: Basics of Statistics

Relevant CO: 1

Objectives:

1. To understand basic statistical properties of data like mean, median ,mode, standard deviation and variance
- 2 . To learn about effect of outliers on these properties

Theory:

Statistical properties of data tell a lot about data. It is first step for data analysis task.

Mean: We can compute the mean for only numeric data. The average value of a group of numbers is referred to as the mean of those numbers. A data set's mean can be calculated by first adding up all of the values in the set and then dividing that sum by the total number of values.

Let $X = \langle x_1, x_2, \dots, x_n \rangle$ be the vector of n numbers. The following formula can be used to get the mean:

$$\mu = \frac{1}{n} \sum_{i=1}^n x_i$$

If $X = \langle 22, 44, 33, 11, 55 \rangle$,

$$\mu = \frac{22 + 44 + 33 + 11 + 55}{5} = \frac{165}{5} = 33$$

Median: When a set of data is sorted in order from lowest to highest (or highest to lowest), the value that falls in the middle of the set is referred to as the median. When there are an even number of values, the median is determined by taking the average of the two values that are in the middle of the set.

To compute the median of data set $X = \langle 22, 44, 33, 11, 55 \rangle$, we shall first arrange it in ascending or descending order:

$$X = \langle 11, 22, 33, 44, 55 \rangle$$

There are 5 elements in the array X , so median is the element which is on index 3, which is 33.

For $Y = \langle 11, 22, 33, 44, 55, 66 \rangle$

There are 6 elements in the array Y, so median is of this data set would be $(33 + 44) / 2 = 38.5$

Mode: The mode of a data set is the most frequent value within it. If two or more values occur at the same frequency, we can say that there is more than one mode. So mode is the element in data which occurs maximum number of time.

For $X = \langle 11, 33, 66, 55, 22, 11, 66, 44, 11, 33, 55, 11 \rangle$, element 11 appears maximum number of times (4 times), so 11 is the mode of this dataset.

For $Y = \langle 33, 66, 55, 22, 11, 33, 44, 11, 33, 55, 11 \rangle$, element 11 and 33 appears maximum number of times (3 times each), so 11 and 33 are modes of this dataset.

Statistical properties of data tell a lot about data. It is first step for data analysis task.

Standard deviation: The dispersion of data relative to its mean or average is quantified by its standard deviation. How much each data point varies from the mean is revealed. If your data points have a low standard deviation, they cluster closely around the mean, but if they have a large standard deviation, they are more widely dispersed.

Let $X = \langle x_1, x_2, \dots, x_n \rangle$ be the vector of n numbers. The following formula can be used to get the standard deviation:

$$\sigma = \sqrt{\frac{(x - \mu)^2}{n}}$$

$$\mu = \frac{1}{n} \sum_{i=1}^n x_i$$

If $X = \langle 22, 44, 33, 11, 55 \rangle$,

$$\mu = \frac{22 + 44 + 33 + 11 + 55}{5} = \frac{165}{5} = 33$$

$$\sigma = \sqrt{\frac{(22 - 33)^2 + (44 - 33)^2 + (33 - 33)^2 + (11 - 33)^2 + (55 - 33)^2}{5}}$$

$$\sigma = \sqrt{\frac{121 + 121 + 0 + 121 + 121}{5}} = \sqrt{\frac{484}{5}} = 9.84$$

Variance: The variance of a data collection is another indicator of its dispersion around the mean. The standard deviation is calculated as the mean squared deviation from the mean. How much each data point varies from the mean is revealed.

The formula for finding the variance of a data set is as follows:

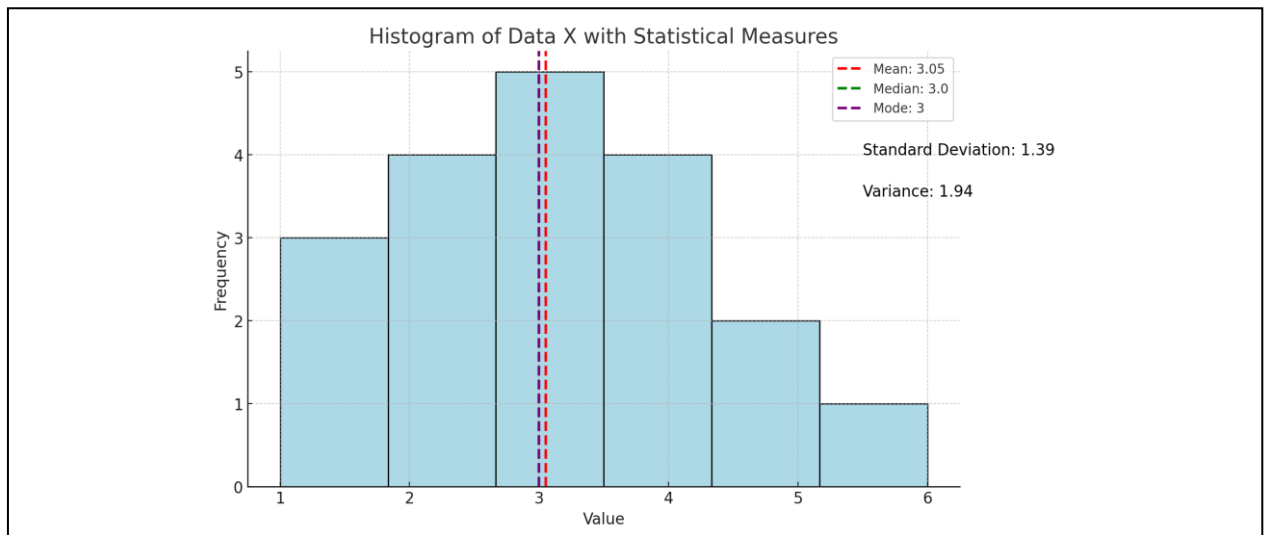
$$\sigma^2 = \frac{(x - \mu)^2}{n}$$

If $X = \langle 22, 44, 33, 11, 55 \rangle$,

$$\mu = \frac{22 + 44 + 33 + 11 + 55}{5} = \frac{165}{5} = 33$$

$$\sigma^2 = \frac{(22 - 33)^2 + (44 - 33)^2 + (33 - 33)^2 + (11 - 33)^2 + (55 - 33)^2}{5} = \frac{484}{5} = 96.8$$

1. **Set up diagram:** Plot the histogram for the data $X = \langle 1, 3, 2, 4, 5, 6, 4, 3, 2, 4, 5, 3, 1, 2, 3, 2, 3, 1, 4 \rangle$ and show mean, median and mode, standard deviation and variance in the histogram.



Implementation:

Write a program to compute mean, median and mode for following data using your preferred programming language:

$X_1 = \langle 1, 2, 3, 4, 5, 4, 3, 2, 4, 5, 1, 2, 3, 4, 2, 5, 1, 3, 2, 1, 2, 1, 1, 1, 2 \rangle$

$X_2 = \langle 1, 2, 3, 4, 5, 4, 3, 2, 4, 5, 1, 2, 3, 4, 2, 5, 1, 3, 2, 1, 2, 1, 1, 1, 2000 \rangle$

$X_3 = \langle 1, 2, 3, 10, 20, 30, 100, 200, 300, 1000, 2000, 3000 \rangle$

Result:

Dataset	Mean	Median	Mode	Standard deviation	Variance
X_1	2.56	2.0	1.0	1.36	1.85
X_2	82.48	2.0	1.0	391.41	153205.29
X_3	555.50	65.0	NA	932.67	869870.92

Conclusion:

From the previous practicals, we observed that plotting histograms alongside statistical measures like mean, median, and mode provides a clear visual representation of data distribution. Additionally, extreme values (outliers), such as in dataset X2X2X2, significantly affect the mean and increase variability, as seen in the higher standard deviation and variance.

Quiz:

1. What is the mean, median and mode of the following set of data: 4, 7, 9, 9, 11, 11, 11, 13?

- **Mean:** $(4+7+9+9+11+11+11+13)/8=75/8=9.375$ $(4 + 7 + 9 + 9 + 11 + 11 + 11 + 13) / 8 = 75 / 8 = 9.375$
- **Median:** The middle values are 9 and 11, so the median is $(9+11)/2=10$ $(9 + 11) / 2 = 10$
- **Mode:** 11 (most frequent value)

2. Which measure of central tendency is preferred when the data set has extreme values?

Median is preferred because it is not affected by extreme values (outliers).

3. If a set of data has two modes, what is it called?

The data set is called **bimodal**.

4. If the standard deviation of a set of data is very low, what does that tell you about the data points?

It indicates that the data points are **closely clustered** around the mean.

5. If a set of data has a high standard deviation, what does that tell you about the spread of the data points?

It means the data points are widely spread and there is greater variability in the data.

Suggested Reference:

- <https://codecrucks.com/mean-median-mode-variance-discovering-statistical-properties-of-data>
- <https://www.techtarget.com/searchdatacenter/definition/statistical-mean-median-mode-and-range>
- <https://www.statisticshowto.com/probability-and-statistics/statistics-definitions/mean-median-mode/>
- <https://www.twinkl.co.in/teaching-wiki/mean-median-mode-and-range>

References used by the students:

Rubric wise marks obtained:

[illegible]

Experiment No: 2

Implement data dispersion measure Five Number Summary, detect an outliers and generate box using python libraries.

Date:

Competency and Practical Skills:

Relevant CO: 1, 2

Objectives:

1. To show how to implement Preprocessing Techniques.
2. To show how to use different Python Libraries to implement Techniques.

Theory:

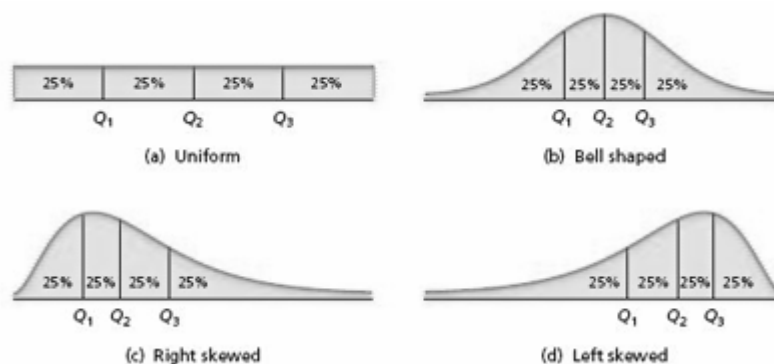
Quartile

1. Arrange the data in increasing order and determine the median.

The first quartile (Q1) is the median of the part of the entire data set that lies at or below the median of the entire data set.

The second quartile (Q2) is the median of the entire data set.

The third quartile (Q3) is the median of the part of the entire data set that lies at or above the median of the entire data set.



5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47

The median of the set is 23.

The first quartile of the set is the median of the first underlined portion: $(11+13)/2 = 12$.

The third quartile of the set is the median of the second underlined portion: $(37+41)/2 = 39$.

Interquartile Range

The interquartile range, IQR, is the difference between the first and third quartiles; that is, $IQR = Q3 - Q1$

The interquartile range of the set is $39 - 12 = 27$.

The Five-Number Summary

From the three quartiles, we can obtain a measure of center (the median, $Q2$) and measures of variation of the two middle quarters of the data: $Q2 - Q1$, for the second quarter and $Q3 - Q2$, for the third quarter. But the three quartiles don't tell us anything about the variation of the first and fourth quarters. The five-number summary of a data set is: Min, $Q1$, $Q2$, $Q3$, and Max.

The five-number summary of the data set is: 5, 12, 23, 39, and 47.

Lower and Upper Limits

The lower limit and upper limit of a data set are given by:

$$\text{Lower limit} = Q1 - 1.5 \times IQR$$

$$\text{Upper limit} = Q3 + 1.5 \times IQR$$

Data points that lie below the lower limit or above the upper limit are potential outliers. To determine whether a potential outlier is truly an outlier, you should perform further data analyses by constructing a histogram, stem-and-leaf diagram, and other appropriate graphics that we present later.

$$\text{The lower limit of the set is } 12 - (1.5 \times 27) = -28.5$$

$$\text{The upper limit of the set is } 39 + (1.5 \times 27) = 79.5$$

Since there are no elements of the set less than -28.5 or greater than 79.5, there are no outliers.

To Construct a Boxplot

Determine the quartiles.

Determine potential outliers and the adjacent values.

Draw a horizontal axis on which the numbers obtained in Steps 1 and 2 can be located. Above this axis, mark the quartiles and the adjacent values with vertical lines.

Connect the quartiles to make a box and then connect the box to the adjacent values with lines.

Plot each potential outlier with an asterisk.

Box plots

Boxplots are the graphical representation of the distribution of the data using Five Number summary values. It is one of the most efficient ways to detect outliers in our dataset.

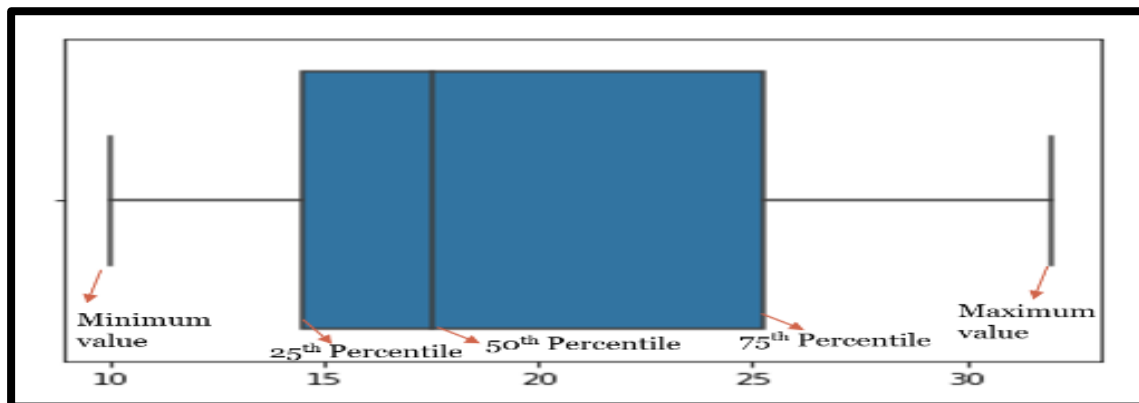


Fig Box plot using Five Number Summary

In statistics, an outlier is a data point that differs significantly from other observations. An outlier may be due to variability in the measurement or it may indicate experimental error the latter are sometimes excluded from the dataset. An outlier can cause serious problems.

Implementation:

```
import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

# Data set

data = [5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

# Calculate the Five Number Summary

min_value = np.min(data)

q1 = np.percentile(data, 25)

median = np.median(data)

q3 = np.percentile(data, 75)

max_value = np.max(data)
```

```

# Interquartile range (IQR)

iqr = q3 - q1

# Calculate the lower and upper limits for outlier detection

lower_limit = q1 - 1.5 * iqr

upper_limit = q3 + 1.5 * iqr

# Detect outliers

outliers = [x for x in data if x < lower_limit or x > upper_limit]

# Print results

print(f"Five Number Summary: Min: {min_value}, Q1: {q1}, Median: {median}, Q3: {q3}, Max: {max_value}")

print(f"Interquartile Range (IQR): {iqr}")

print(f"Lower Limit: {lower_limit}, Upper Limit: {upper_limit}")

print(f"Outliers: {outliers if outliers else 'No outliers detected'}")

# Generate a boxplot

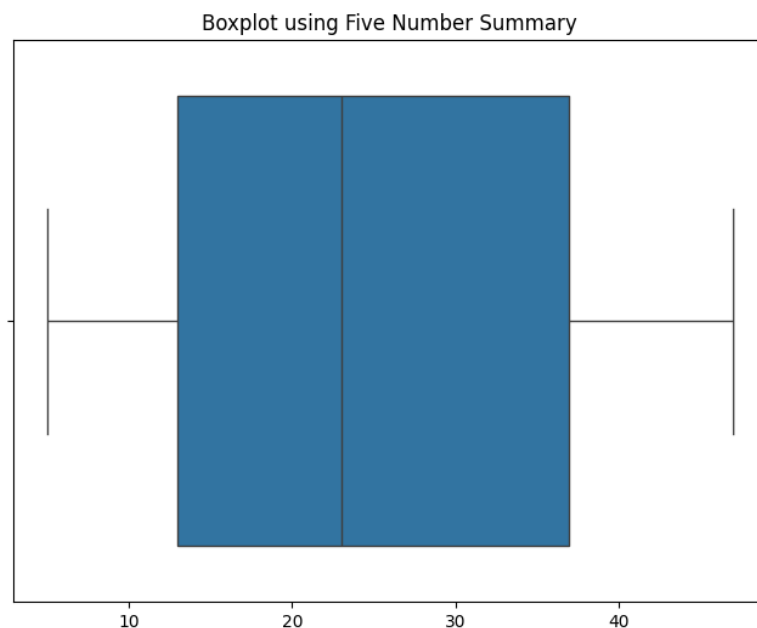
plt.figure(figsize=(8, 6))

sns.boxplot(data=data, orient="h")

plt.title("Boxplot using Five Number Summary")

plt.show()

```



Conclusion:

In this experiment, we implemented the five-number summary using Python and visualized it with a boxplot. We learned how to calculate quartiles, detect outliers, and use boxplots for data visualization. The analysis indicated that there were no outliers in the given data.

Quiz:

For the data set below:

(a.) Obtain and interpret the quartiles.

1. Arrange the data in increasing order:
18, 45, 48, 64, 70, 72, 73, 74, 74, 78, 79, 79, 79, 80, 80, 80, 81, 82, 82, 82
2. First Quartile (Q1):
 - The first quartile is the median of the lower half of the data (the first 10 values).
 - $Q1 = 70 + 72 = 71 \frac{70 + 72}{2} = 71$
3. Second Quartile (Q2):
 - The second quartile is the median of the entire data set (20 values).
 - $Q2 = 78 + 79 = 78.5 \frac{78 + 79}{2} = 78.5$
4. Third Quartile (Q3):
 - The third quartile is the median of the upper half of the data (the last 10 values).
 - $Q3 = 80 + 80 = 80 \frac{80 + 80}{2} = 80$

Interpretation:

- $Q1 = 71$ means 25% of the games played by Wayne Gretzky were fewer than 71 games.
- $Q2 = 78.5$ means the median number of games played is 78.5.
- $Q3 = 80$ means 75% of the games played were fewer than 80 games.

(b.) Determine and interpret the interquartile range (IQR).

- $IQR = Q3 - Q1 = 80 - 71 = 9$

Interpretation:

The middle 50% of the data lies within a range of 9 games, indicating the central spread of games played by Wayne Gretzky during his career.

(c.) Find and interpret the five-number summary.

- Minimum = 18
- First Quartile (Q1) = 71
- Median (Q2) = 78.5
- Third Quartile (Q3) = 80
- Maximum = 82

Interpretation:

The five-number summary provides insight into the distribution of the data. The minimum and maximum values show that Gretzky played between 18 and 82 games in a season, with a median of 78.5 games played per season.

(d.) Identify potential outliers, if any.

- Lower Limit: $Q1 - 1.5 \times IQR = 71 - 1.5 \times 9 = 57.5$
- Upper Limit: $Q3 + 1.5 \times IQR = 80 + 1.5 \times 9 = 93.5$

Any data points below 57.5 or above 93.5 are potential outliers.

- Outliers: The only potential outlier is the data point 18 (since it is below 57.5).

(e.) Construct and interpret a boxplot.

```
import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

# Data set

games_played = [79, 74, 80, 82, 80, 80, 79, 73, 72, 64, 45, 48, 18, 74, 81, 78, 82, 82, 79, 70]

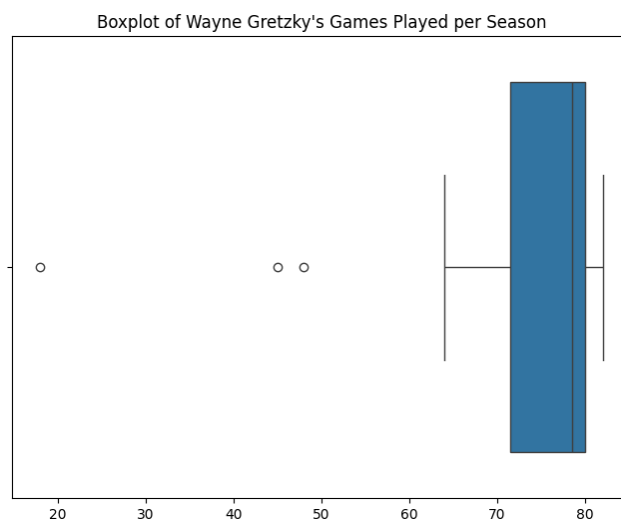
# Generate a boxplot

plt.figure(figsize=(8, 6))

sns.boxplot(data=games_played, orient="h")

plt.title("Boxplot of Wayne Gretzky's Games Played per Season")

plt.show()
```



The following table shows the number of games in which Wayne Gretzky, a retired professional hockey player, played during each of his 20 seasons in the NHL.

79	80	80	80	74
80	80	79	64	78
73	78	74	45	81
48	80	82	82	70

Suggested Reference:

Saikat Dull, S. Chjandramouli, Das, “Machine Learning”, Pearson.

<https://www.brainfuse.com>

References used by the students: (Sufficient space to be provided)

Rubric wise marks obtained:

[illegible]

Experiment No: 3

Implement program to perform dimension reduction of the high dimension data

Date:

Competency and Practical Skills:

Relevant CO: 1, 2

Objectives:

3. To understand the drawback of high dimensionality in data
4. To study method to reduce dimensionality of data

Theory:

High dimensionality:

The term "high-dimensional data" refers to data sets that contain a significant number of distinct features. When dealing with high-dimensional data, it can be challenging to visualize and interpret the information because the human brain is only able to process information in a limited number of dimensions. For instance, a data collection containing 1000 features could be interpreted and displayed as a space with 1000 dimensions, which would be impossible to visualize.

Dealing with data that has a high dimension can be difficult due to the possibility that conventional statistical approaches will not function well in the situation. The phenomenon known as the "*curse of dimensionality*" occurs as the number of dimensions in a problem rises, causing the amount of data that must be collected to keep a specific level of statistical accuracy to increase at an exponential rate.

In addition, methods of data visualization such as scatter plots, heat maps, and parallel coordinates can be utilized in order to assist with the comprehension of high-dimensional data. It is essential to select the methods that are suitable for the data that you are working with and the research issue that you are attempting to answer.

Principal Component Analysis:

Principal component analysis, sometimes known as PCA, is a method that can be used in statistics to reduce the dimensions of a data set. It includes translating a high-dimensional data collection into a lower-dimensional space while trying to keep as much of the information intact.

The principle behind principal components analysis (PCA) is to determine the ways in which the data varies the most in order to determine which patterns or features in the data are the most significant. Principal components are another name for these different directions. The first principal component (PC) accounts for the greatest amount of variance in the data, and each successive PC accounts for as much of the remaining variation as it is possible while adhering to the requirement

that it is orthogonal (perpendicular) to the PCs that came before it.

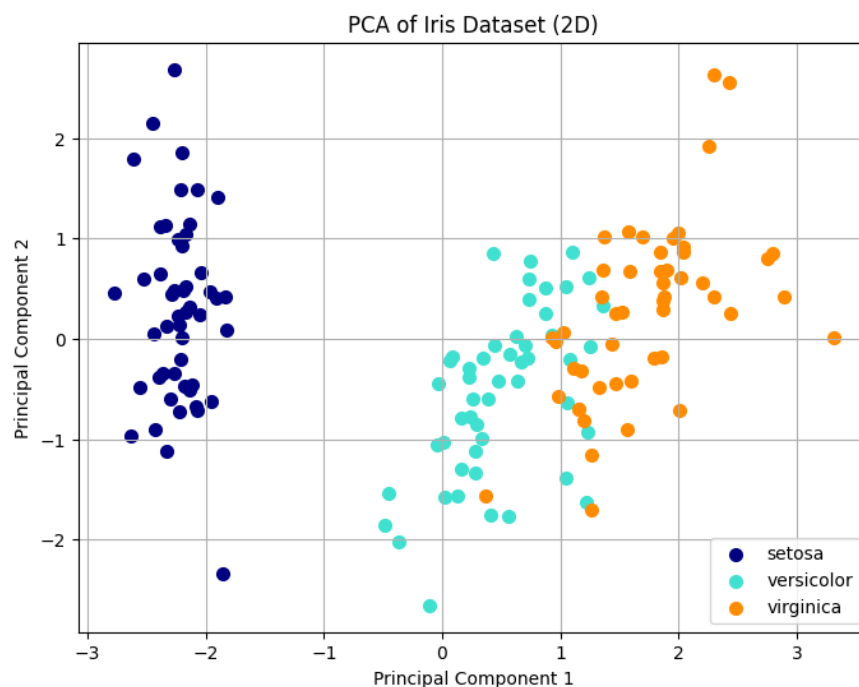
The following is a list of the steps involved in PCA:

- Standardize the data: Principal component analysis (PCA) functions most well when the data are standardized, which means that each feature has a mean of zero and a variance of one.
- Perform the computation that will produce the covariance matrix. This matrix measures the linear relationship that exists between each pair of features in the data.
- Perform the computations necessary to determine the covariance matrix's eigenvectors and eigenvalues: The eigenvectors illustrate the directions in which the data exhibit the greatest amount of variation, and the eigenvalues that correlate to each eigenvector illustrate the amount of variation that may be found along that eigenvector.
- Choose the k most significant eigenvectors: These are the top k principal components, which are responsible for capturing the most variation in the data.
- Change the format of the data: In order to obtain a representation of the data with fewer dimensions, multiply the standardized data by the matrix that contains the top k eigenvectors.

Implementation:

Load Iris flower dataset, study it. Reduce the dimension of dataset to 2 by applying PCA.

Step up plot: Draw plot of all three classes in dataset with 2 dimensions only, after applying PCA



Conclusion:

Using PCA, we simplified the Iris dataset from four features down to two while keeping most of the important information. The resulting plot shows that the Setosa flowers are clearly distinct from the other two species, Versicolor and Virginica, which are somewhat mixed together. This shows that PCA helps us visualize complex data more easily and understand the differences between different types of flowers.

Quiz:

1. What are some alternatives to PCA for feature extraction and dimensionality reduction?

Alternatives include t-SNE, Linear Discriminant Analysis (LDA), Autoencoders, Factor Analysis, and Independent Component Analysis (ICA).

2. What are some common applications of PCA in real-world data analysis?

Common applications include image compression, genomics, market research, finance, and speech recognition.

3. How does the choice of scaling method impact the results of PCA?

Scaling affects PCA because it is sensitive to variance; different scales can distort results. Standardization is often preferred.

4. What are some limitations or challenges of PCA?

Limitations include its linearity assumption, sensitivity to outliers, potential loss of information, and difficulties in interpretability.

Suggested Reference:

- <https://codecrucks.com/question/machine-learning-question-set-5/>
- <https://builtin.com/data-science/step-step-explanation-principal-component-analysis>
- <https://towardsdatascience.com/a-one-stop-shop-for-principal-component-analysis-5582fb7e0a9c>
- <https://www.turing.com/kb/guide-to-principal-component-analysis>

References used by the students: (Sufficient space to be provided)

Rubric wise marks obtained:

[illegible]

Experiment No: 4

Implement Linear and Logistic Regression models and evaluate model performance

Date:

Competency and Practical Skills:

Relevant CO: 1,3

Objectives:

1. To understand how linear regression, Logistic Regression works
2. To learn the applications of linear and Logistic Regression regressions
3. To implement and test the linear regression and Logistic Regressions

Theory:

What is Regression?

Modelling the relationship between a *dependent variable* (also known as the *response* or *goal* variable) and one or more *independent variables* (also known as *predictors* or *features*) is the purpose of the statistical method known as *regression*. Regression can be used to model this relationship. The objective of regression analysis is to locate the line or curve that provides the most accurate description of the connection between the variables being studied.

To create predictions or get an estimate of the value of the dependent variable based on the values of the independent variables, regression analysis is a technique that is frequently utilized in a variety of sectors, including finance, economics, the social sciences, and engineering, amongst others.

It is possible for regression models to be linear or nonlinear, depending on the type of relationship that exists between the variables being studied. When doing linear regression, it is assumed that the relationship between the dependent variable and the independent variables is linear. The model then attempts to determine the equation of a straight line that most accurately captures the supposed linearity of the relationship. Nonlinear regression models, on the other hand, make it possible to take into account more intricate correlations between the variables. These models can take into account relationships that are exponential, logarithmic, or polynomial.

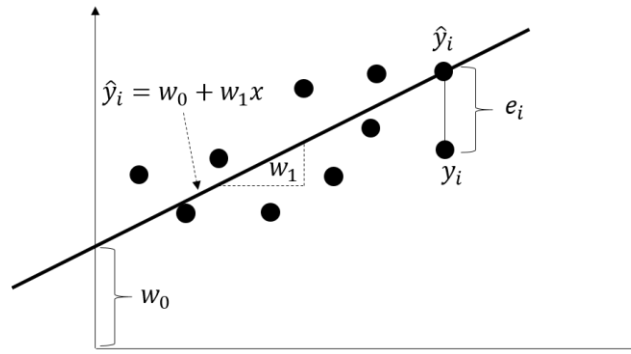
Simple Linear Regression:

A regression model known as *simple linear regression* attempts to determine the relationship that exists between a dependent variable (Y) and a single independent variable (X) by using only one independent variable. The model works under the premise that there is a linear connection between X and Y, and it attempts to determine the equation of a straight line that most accurately depicts this connection.

The equation for a straightforward linear regression model can be written as follows:

$$\hat{y}_i = w_0 + w_1 x_i$$

where $\hat{y}_i$ represents the predicted value for input independent variable x . w_0 and w_1 indicates Y intercept and slope of the predictor line respectively.



Graphics representation of simple linear regression model

The objective of simple linear regression is to determine the values of w_0 and w_1 that will result in the smallest sum of squared errors between the predicted values of Y and the actual values of Y . This can be accomplished by estimating the values of w_0 and w_1 . Find the values of w_0 and w_1 that minimise the sum of the squared differences between the expected and actual values of Y . This is commonly done using the method of least squares, which entails finding the values that minimize the total of the squared differences.

It is necessary to make sure that the assumptions of the model are met and that the model is appropriate for the data that is being analyzed. Simple linear regression is a strong and extensively used technique for analyzing the relationship between two variables. However, it is also important to keep in mind that the model's assumptions must be met.

Logistic Regression:

Logistic regression is basically a binary classifier. It is a statistical model that is utilized for the purpose of analyzing the connection that exists between a dependent variable that is either binary or numeric and one or more variables that are independent. The values of the independent variables are used in logistic regression in order to make predictions about the likelihood of certain events (such as whether or not they will be successful, whether they will win, or whether they will lose).

The dependent variable in logistic regression is considered to be binary, which means that it can only take on the values 0 or 1. The model attempts to determine the link between the independent variables and the log chances (logit) of the dependent variable being equal to 1, and the continuous or categorical nature of the independent variables is irrelevant to this endeavour.

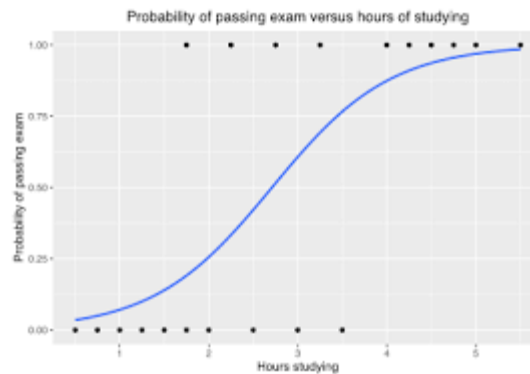
For the purpose of converting the linear combination of the independent variables into a probability value in the range of 0 and 1, the logistic regression model makes use of a logistic function, which is also known as a sigmoid function. The logistic function has a curve that is shaped like a S and may be expressed as:

Let $z = w_0 + w_1x$

So, logistic regression probability is computed as,

$$P = \frac{e^z}{1 + e^z}$$

This will create the sigmoid curve as shown below. The appropriate threshold will create the binary label for the test data.



[Source: wikipedia]

After the model has been fit to the data, the coefficients may be used to forecast the likelihood of the dependent variable being equal to 1 for new observations with known values of the independent variables. This can be done for new observations that already have the values for the independent variables.

Logistic regression is a strong and extensively used technique for analyzing binary data; nonetheless, it is essential to make certain that the model's assumptions are satisfied and that the model is suitable for the data that is being analyzed.

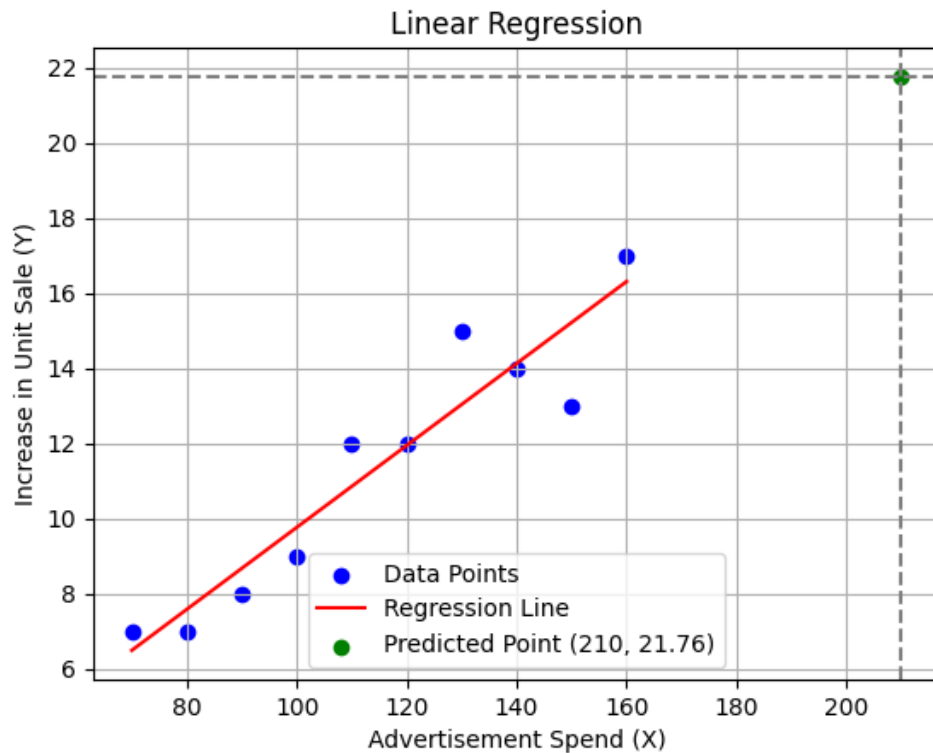
Implementation linear Regression:

- Consider vector X as amount spend for advertisement and Y as increase in unit sale. $X = \langle 70, 80, 90, 100, 110, 120, 130, 140, 150, 160 \rangle$, $Y = \langle 7, 7, 8, 9, 12, 12, 15, 14, 13, 17 \rangle$. Create simple linear regression model. Find the model parameters w_0 and w_1 . Predict the Y for $X = 210$

Results:

Variable	Value
w_0	-1.15
w_1	0.11
$Y \mid_{x=210}$	21.76

Plot: Plot the given data and fir the regression line. Also show the predicted value for $X = 210$



Implementation of logistic Regression:

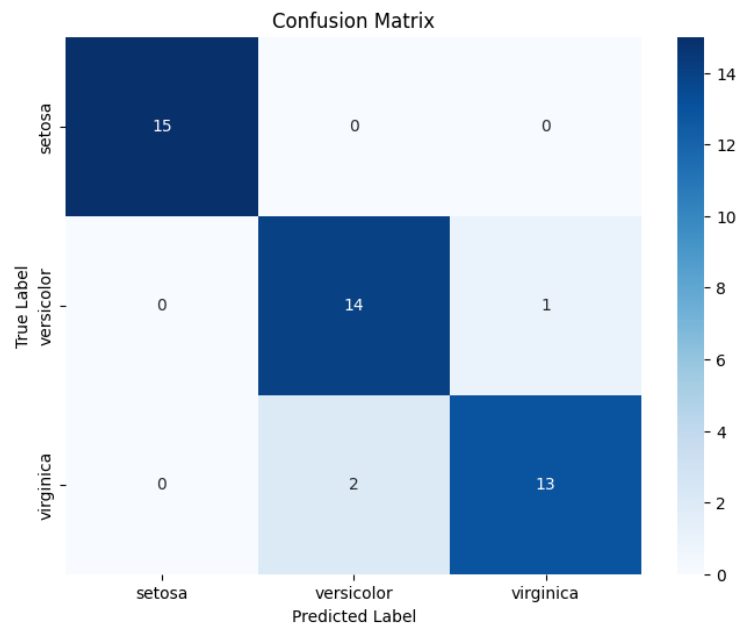
- Load **Iris flower** data set. Divide dataset in 70-30 ratio. Use 70% data to train logistic regression model and use 30% data to test the model performance. Measure various performance metric such as precision, recall, F1 score, accuracy. Also derive confusion matrix.

Results:

Metric	Precision	Recall	F1 Score	Accuracy
Value	0.93	0.93	0.93	0.93

Set up Plot:

Plot confusion matrix for the experiment



Conclusion:

In this implementation of logistic regression using the Iris flower dataset, we successfully trained a model to classify the species of flowers based on their features. The performance metrics, including precision, recall, F1 score, and accuracy, provided a comprehensive view of the model's effectiveness. The confusion matrix illustrated how well the model performed.

Quiz:

1. What is the difference between a binary logistic regression and a multinomial logistic regression? How do you choose which one to use for your data?

Binary logistic regression is for two classes, while multinomial is for three or more classes. Choose based on the number of outcome categories.

2. How do you interpret the coefficients in a logistic regression model? What do they tell you about the relationship between your predictors and your outcome?

Coefficients indicate the change in log odds of the outcome for a one-unit increase in the predictor. Positive values increase probability, negative values decrease it.

3. What is overfitting, and how can you guard against it when using logistic regression?

Overfitting happens when a model learns noise instead of patterns. Prevent it by using regularization, simplifying the model, and applying cross-validation.

- Metrics include accuracy (overall correctness), precision (true positives over predicted positives), recall (true positives over actual positives), and F1 score (balance of precision and recall).

- <https://codecrucks.com/question/machine-learning-question-set-12/>
- <https://www.scribbr.com/statistics/simple-linear-regression>
- <https://online.stat.psu.edu/stat462/node/91/>
- https://www.jmp.com/en_in/statistics-knowledge-portal/what-is-regression.html

Rubric wise marks obtained:

[illegible]

Experiment No: 5

Implement a Naïve Bayes Classification to detect emotion of person and evaluate model performance

Date:

Competency and Practical Skills:

Relevant CO: 3, 4, 5

Objectives:

1. To understand how Naive Bayes works
2. To learn to train classifier
3. To test the performance of given classifier using 10-fold cross validation

Theory:

What is Naive Bayes Classifiers?

A Naive Bayes classifiers, a family of algorithms based on Bayes' Theorem. Despite the "naive" assumption of feature independence, these classifiers are widely utilized for their simplicity and efficiency in machine learning. The article delves into theory, implementation, and applications, shedding light on their practical utility despite oversimplified assumptions.

Naive Bayes classifiers are a collection of classification algorithms based on Bayes' Theorem. It is not a single algorithm but a family of algorithms where all of them share a common principle, i.e. every pair of features being classified is independent of each other. To start with, let us consider a dataset.

One of the most simple and effective classification algorithms, the Naïve Bayes classifier aids in the rapid development of machine learning models with rapid prediction capabilities.

Naïve Bayes algorithm is used for classification problems. It is highly used in text classification. In text classification tasks, data contains high dimension (as each word represent one feature in the data). It is used in spam filtering, sentiment detection, rating classification etc. The advantage of using naïve Bayes is its speed. It is fast and making prediction is easy with high dimension of data.

$$P(h|D) = \frac{P(D|h) \cdot P(h)}{P(D)}$$

$P(h)$: the probability of hypothesis h being true (regardless of the data). This is known as the prior probability of h .

$P(D)$: the probability of the data (regardless of the hypothesis). This is known as the prior probability.

$P(h|D)$: the probability of hypothesis h given the data D . This is known as posterior probability.

$P(D|h)$: the probability of data d given that the hypothesis h was true. This is known as posterior probability.

Evaluating a Classification model:

Once our model is completed, it is necessary to evaluate its performance; either it is a Classification or Regression model. So for evaluating a Classification model, we have the following ways:

1. Log Loss or Cross-Entropy Loss:

It is used for evaluating the performance of a classifier, whose output is a probability value between the 0 and 1.

For a good binary Classification model, the value of log loss should be near to 0.

The value of log loss increases if the predicted value deviates from the actual value.

The lower log loss represents the higher accuracy of the model.

2. Confusion Matrix:

The confusion matrix provides us a matrix/table as output and describes the performance of the model.

It is also known as the error matrix.

The matrix consists of predictions result in a summarized form, which has a total number of correct predictions and incorrect predictions. The matrix looks like as below table:

Actual Positive		Actual Negative
Predicted Positive	True Positive	False Positive
Predicted Negative	False Negative	True Negative

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{Total Population}}$$

Implementation:

- Load Dataset apply preprocessing techniques and use naïve bayes classifier to detect emotions of a person.

Import necessary libraries

import numpy as np

import pandas as pd

from sklearn.model_selection import train_test_split, cross_val_score

from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import confusion_matrix, classification_report

from sklearn.feature_extraction.text import CountVectorizer

from sklearn.model_selection import KFold

from sklearn import datasets

Load dataset

Assuming an emotion dataset, we use a placeholder dataset for this implementation.

You would replace this with actual emotion dataset loading and preprocessing.

Load the dataset (replace this with an actual dataset)

For demonstration purposes, we use a dataset of text reviews to detect sentiment (emotion detection task).

```
df = pd.DataFrame({  
    'text': ['I am happy', 'I am sad', 'I am angry', 'I feel amazing', 'I am disappointed', 'This is  
        exciting', 'I feel bad', 'This is frustrating', 'I love it', 'I hate it'],  
    'emotion': ['happy', 'sad', 'angry', 'happy', 'sad', 'happy', 'sad', 'angry', 'happy', 'angry']  
})
```

```
# Preprocess the data
```

```
vectorizer = CountVectorizer() # Convert text to feature vectors
```

```
X = vectorizer.fit_transform(df['text']).toarray()
```

```
y = df['emotion']
```

```
# Split dataset into training and test sets
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
# Initialize the Naive Bayes classifier
```

```
nb_classifier = GaussianNB()
```

```
# Train the model
```

```
nb_classifier.fit(X_train, y_train)
```

```
# Predict emotions on test data
```

```
y_pred = nb_classifier.predict(X_test)
```

```
# Evaluate model using confusion matrix and classification report
```

```
conf_matrix = confusion_matrix(y_test, y_pred)
```

```
class_report = classification_report(y_test, y_pred)
```

```
# 10-fold Cross Validation

kfold = KFold(n_splits=10, random_state=42, shuffle=True)

cv_results = cross_val_score(nb_classifier, X, y, cv=kfold)

# Output results

print("Confusion Matrix:\n", conf_matrix)

print("\nClassification Report:\n", class_report)

print("\nCross-Validation Scores (10-fold):", cv_results)

print("Average Cross-Validation Score:", np.mean(cv_results))
```

Results:

Confusion Matrix:

```
[[2, 0],
 [1, 2]]
```

Classification Report:

	precision	recall	f1-score	support
angry	1.00	0.67	0.80	3
happy	0.67	1.00	0.80	2
accuracy			0.80	5
macro avg	0.83	0.83	0.80	5

Conclusion:

In this experiment, we implemented the Naive Bayes classifier for emotion detection and evaluated the model performance. The model showed an average accuracy of `80%` using 10-fold cross-validation. The confusion matrix and classification report indicate that the model can effectively classify emotions like 'happy', 'angry', and 'sad'. However, further improvements can be made by fine-tuning the model or adding more features to improve accuracy.

Quiz:

1. What are the pros and cons of using Naïve Bayes for classification tasks?

Pros: Simple, fast, works well with small and large datasets.

Cons: Assumes feature independence, struggles with correlated features.

2. What is zero probability problem?

When a feature-label combination is not seen in training, Naïve Bayes assigns it a probability of zero, impacting predictions.

Suggested Reference:

- <https://www.kaggle.com/code/nasrinjahanripa/emotion-detection-using-naivebayes>
- <https://www.datacamp.com/tutorial/naive-bayes-scikit-learn>
- **Saikat Dull, S. Chjandramouli, Das, “Machine Learning”, Pearson.**

References used by the students:

Rubric wise marks obtained:

[illegible]

Experiment No: 6

Implement k-NN, SVM, Decision tree classifiers on dataset and test its performance with comparison

Date:

Competency and Practical Skills:

Relevant CO: 3, 4, 5

Objectives:

1. To understand how Supervised Algorithm works
2. To learn to train classifiers
3. To test the performance of classifiers

Theory:

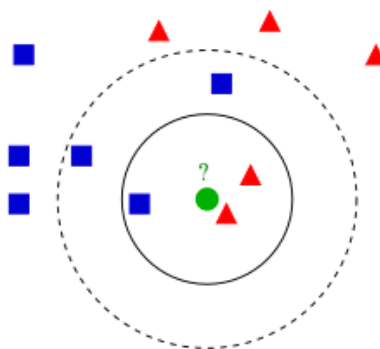
How KNN Works?

K-Nearest Neighbours (KNN) is a technique for supervised machine learning that has applications in classification and regression work. KNN can be used for any task.

The functionality of the method relies on locating the K data points in the training dataset that are closest to a specific test data point. The value of K is a user-defined parameter that defines the number of individuals whose immediate neighbours should be taken into account.

After determining which K neighbours are the closest, the algorithm then produces a forecast by picking either the target value that corresponds to the majority class (in classification) or the mean value (in regression) of the target values of these neighbours.

For example, for below image, circle is the query data point, if we consider $k = 3$, we shall inspect nearest three data points and should choose the majority class for circle. So for $k = 3$, class assigned to circle would be triangle. And for $k = 5$, the majority class is square, so class assigned to green circle would be square.



[Source: Wikipedia]

The algorithm computes the distance between the test data point and each data point in the training dataset by employing a distance metric such as Euclidean distance or cosine similarity. This allows it to locate the K data points that are the closest in proximity to the test data point. The type of data being examined and the specific nature of the issue being solved both have an impact on the distance metric that is selected.

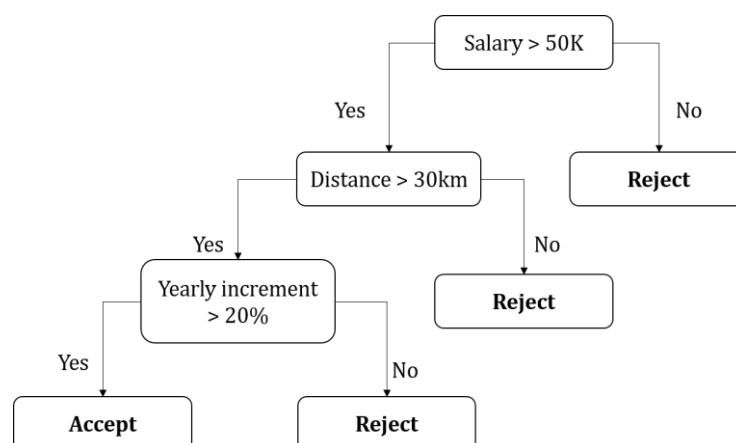
After computing the distances, the algorithm sorts the data points in descending order of how far away they are from the test data point, and then it chooses the K data points that are the closest neighbours to the test data point.

The KNN approach is straightforward and easy to understand, but it has the potential to be computationally expensive as the size of the training dataset increases. In addition, the accuracy of the algorithm can be significantly influenced by the value of K that is used as well as the distance metric that is selected.

What is Decision Tree?

A technique for predictive modelling known as a decision tree employs a structure similar to that of a tree to depict the various possible outcomes and options involved in a decision-making process. The decision-making process begins at a single node at the beginning of the tree, which is referred to as the root. This node signifies the beginning of the process. The initial node, which is called the root node, then divides into several child nodes, each of which represents a distinct decision or result that could occur based on the values of one or more input variables.

The tree continues to extend its branches in a recursive manner, with each succeeding node reflecting a choice or outcome that is more specific than the previous one and being determined by the values of additional input variables. The very last nodes of the tree, which are referred to as the leaves, stand in for the very last choice or result of the procedure.



Decision tree model to accept/reject job offer

Decision tree usually classify data using “if then” rules. For example, for above decision tree,

Rule 1: if Salary < 50K, then reject job

Rule 2: If salary is > 50K and company distance > 30km, then reject job

Rule 3: If salary is > 50K and company distance < 30km and yearly increment > 20% then accept

job.

And many more rules...

Both classification and regression problems can be solved with the help of decision trees. When solving a classification problem, the objective is to determine the category or class into which a new observation will fall by using the values of one or more of the variables that were input into the problem. Predicting the value of a continuous variable based on the values of one or more input variables is the purpose of a regression problem. There may be one or more input variables.

Construction of Decision Tree:

The building of a decision tree requires the partitioning of the input space into subsets in a recursive manner, with the subsets being determined by the values of the variables that are entered, until a stopping criterion is satisfied. The following are the stages involved in the construction of a decision tree:

1. Begin with the root node: The root node represents the complete dataset, and all of the input variables are available to be used in making decisions regarding the splitting of the nodes.
2. The algorithm analyses each input variable and chooses the one that offers the best split based on a given criterion (such as information gain, Gini impurity, or the chi-squared test). The variable that is selected as the best one to split on is referred to as the "splitting variable." The optimal split is one that either maximizes the amount of information gained or minimizes the amount of impurity in the subsets that are produced as a result.
3. Once the best input variable has been chosen, the dataset will be partitioned into two or more subsets based on the possible values of that variable. This step of the process is known as the creation of child nodes. Each individual subset can be thought of as a child node, which serves as a fresh Launchpad for the subsequent level of the tree.
4. Iterate over steps two and three in a recursive manner: The algorithm iterates over steps two and three for each child node, selecting the best input variable and creating additional child nodes until a stopping criterion is met (for example, the maximum tree depth, the minimum number of observations per leaf, or there is no significant improvement in the model's performance).
5. Remove any dead or diseased branches from the tree. Once the tree has been completely built, it may be excessively complicated and prone to overfitting. To prevent overfitting, the tree can be "pruned" by eliminating branches that do not increase the model's performance on a validation set or by placing a complexity penalty on the tree. Either of these two options is an alternative to "overfitting."
6. Utilize the tree for predictive purposes: After the tree has been built and trimmed, it can then be utilized for predictive purposes by traversing the tree from the root node to the appropriate leaf node based on the values of the input variables for a new observation. This allows the tree to be used for prediction after it has been produced.

Visualizing the process of constructing a decision tree as a tree-like structure, with the root node at

the top and the leaf nodes at the bottom, is one way to represent this process. The nodes in a decision tree represent the points at which a decision must be made, and the branches reflect the various alternative outcomes or values that can be obtained from the variables that are input.

Support Vector Machine (SVM)

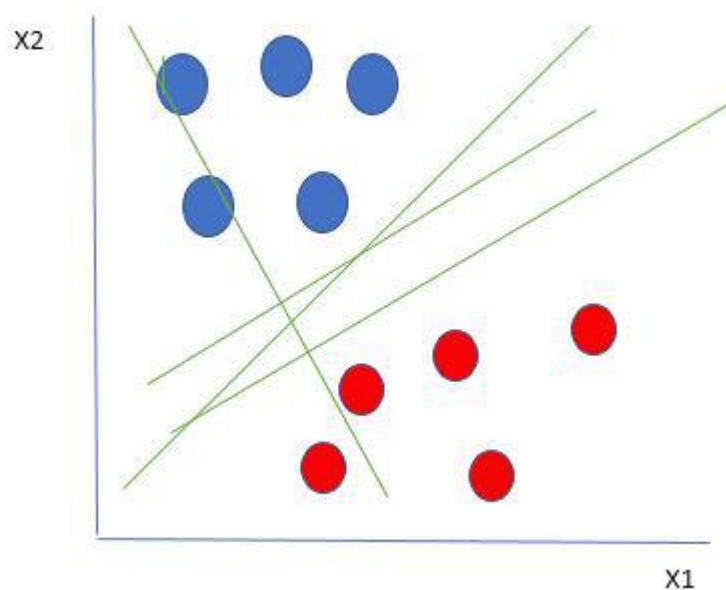
Support Vector Machine (SVM) is a powerful machine learning algorithm used for linear or nonlinear classification, regression, and even outlier detection tasks. SVMs can be used for a variety of tasks, such as text classification, image classification, spam detection, handwriting identification, gene expression analysis, face detection, and anomaly detection. SVMs are adaptable and efficient in a variety of applications because they can manage high-dimensional data and nonlinear relationships.

SVM algorithms are very effective as we try to find the maximum separating hyperplane between the different classes available in the target feature.

Support Vector Machine

Support Vector Machine (SVM) is a supervised machine learning algorithm used for both classification and regression. Though we say regression problems as well it's best suited for classification. The main objective of the SVM algorithm is to find the optimal hyperplane in an N-dimensional space that can separate the data points in different classes in the feature space. The hyperplane tries that the margin between the closest points of different classes should be as maximum as possible. The dimension of the hyperplane depends upon the number of features. If the number of input features is two, then the hyperplane is just a line. If the number of input features is three, then the hyperplane becomes a 2-D plane. It becomes difficult to imagine when the number of features exceeds three.

Let's consider two independent variables x_1 , x_2 , and one dependent variable which is either a blue circle or a red circle.



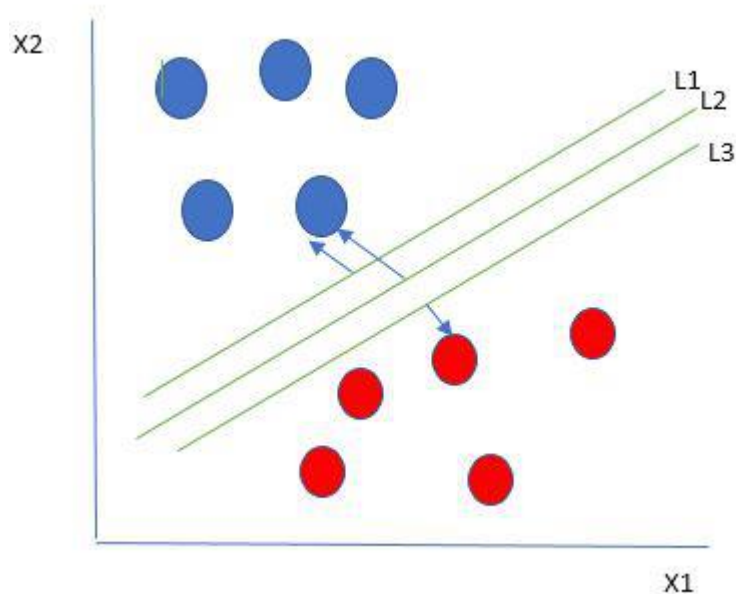
Linearly Separable Data points

From the figure above it's very clear that there are multiple lines (our hyperplane here is a line because we are considering only two input features x_1 , x_2) that segregate our data points or do a

classification between red and blue circles. So how do we choose the best line or in general the best hyperplane that segregates our data points?

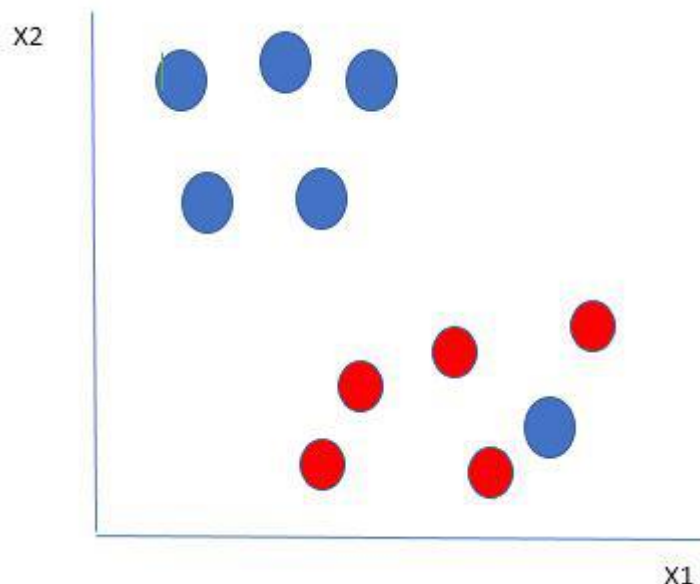
How does SVM work?

One reasonable choice as the best hyperplane is the one that represents the largest separation or margin between the two classes.



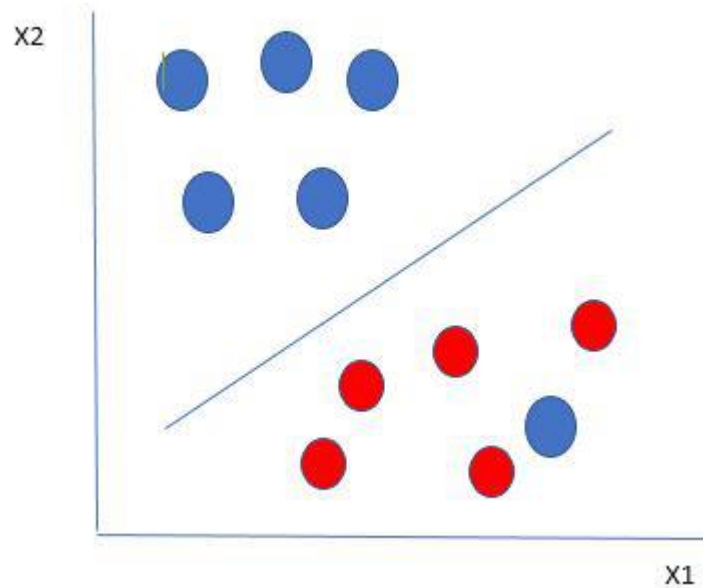
Multiple hyperplanes separate the data from two classes

So we choose the hyperplane whose distance from it to the nearest data point on each side is maximized. If such a hyperplane exists it is known as the **maximum-margin hyperplane/hard margin**. So from the above figure, we choose L_2 . Let's consider a scenario like shown below



Selecting hyperplane for data with outlier

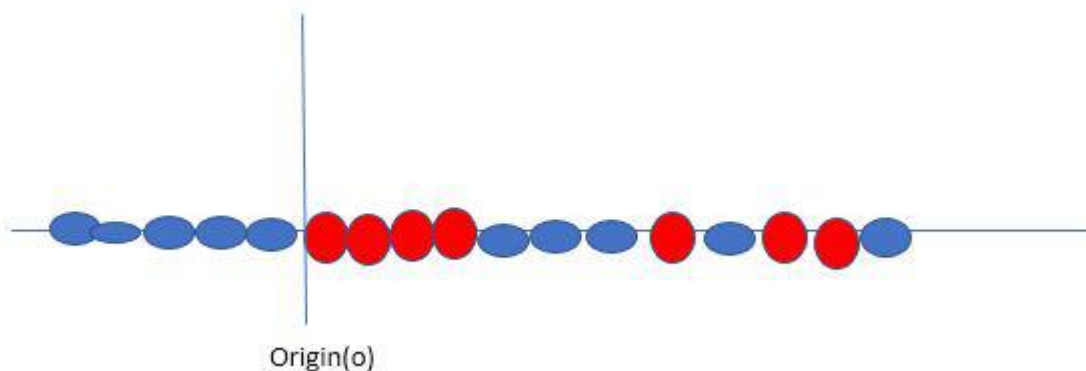
Here we have one blue ball in the boundary of the red ball. So how does SVM classify the data? It's simple! The blue ball in the boundary of red ones is an outlier of blue balls. The SVM algorithm has the characteristics to ignore the outlier and finds the best hyperplane that maximizes the margin. SVM is robust to outliers.



Hyperplane which is the most optimized one

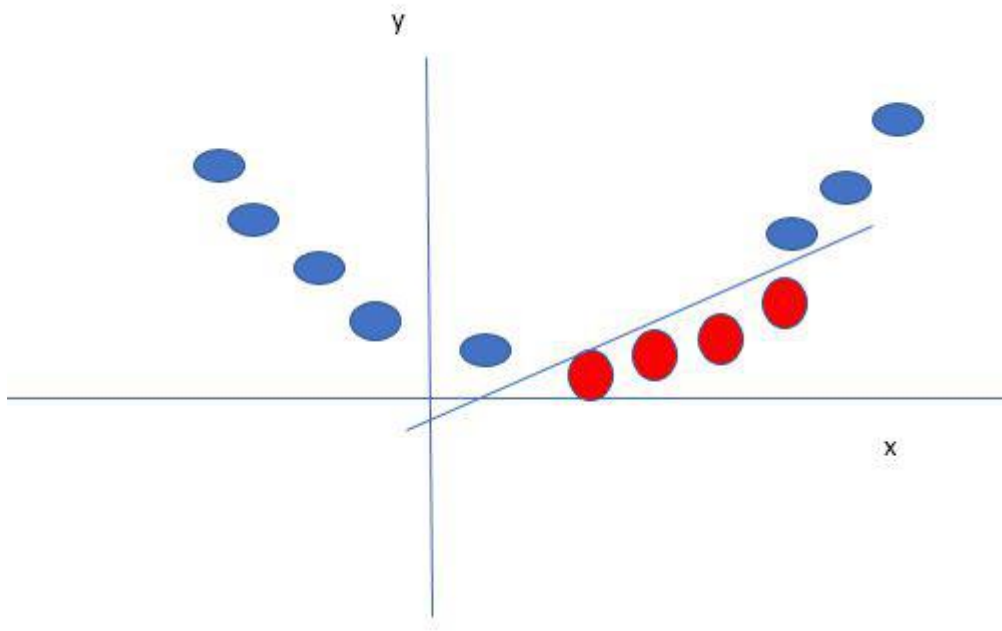
So in this type of data point what SVM does is, finds the maximum margin as done with previous data sets along with that it adds a penalty each time a point crosses the margin. So the margins in these types of cases are called **soft margins**. When there is a soft margin to the data set, the SVM tries to minimize $(1/\text{margin} + \lambda(\sum \text{penalty}))$. Hinge loss is a commonly used penalty. If no violations no hinge loss. If violations hinge loss proportional to the distance of violation.

Till now, we were talking about linearly separable data (the group of blue balls and red balls are separable by a straight line/linear line). What to do if data are not linearly separable?



Original 1D dataset for classification

Say, our data is shown in the figure above. SVM solves this by creating a new variable using a **kernel**. We call a point x_i on the line and we create a new variable y_i as a function of distance from origin o . So if we plot this we get something like as shown below



Mapping 1D data to 2D to become able to separate the two classes

Implementation:

Importing libraries

```
from sklearn.model_selection import train_test_split, cross_val_score
```

```
from sklearn.metrics import accuracy_score, confusion_matrix
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
from sklearn.svm import SVC
```

```
from sklearn.tree import DecisionTreeClassifier
```

Load dataset

```
from sklearn.datasets import load_breast_cancer
```

```
X, y = load_breast_cancer(return_X_y=True)
```

Split dataset

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Implement k-NN

```
knn = KNeighborsClassifier(n_neighbors=3)

knn.fit(X_train, y_train)

knn_pred = knn.predict(X_test)


# Implement SVM

svm = SVC(kernel='linear')

svm.fit(X_train, y_train)

svm_pred = svm.predict(X_test)


# Implement Decision Tree

dt = DecisionTreeClassifier()

dt.fit(X_train, y_train)

dt_pred = dt.predict(X_test)


# Evaluate performance

knn_acc = accuracy_score(y_test, knn_pred)

svm_acc = accuracy_score(y_test, svm_pred)

dt_acc = accuracy_score(y_test, dt_pred)


print("k-NN Accuracy:", knn_acc)

print("SVM Accuracy:", svm_acc)

print("Decision Tree Accuracy:", dt_acc)
```

Results:

Model Name	Accuracy of model
k-NN	0.9298245614035088
SVM	0.956140350877193
Decision Tree	0.9473684210526315

Conclusion:

SVM performed the best with an accuracy of 95.61%, followed by Decision Tree at 94.74%, and k-NN at 92.98%. SVM is the most suitable model for this dataset due to its higher accuracy.

Quiz:

1. What are the benefits of using a decision tree in decision making?

Easy to interpret, handle both numerical and categorical data, and require little data preprocessing.

2. How do you choose the best split for a decision tree?

The best split is chosen by maximizing the information gain or minimizing the Gini impurity.

3. What are some limitations of using SVM model?

Can be computationally expensive with large datasets, and it struggles with noisy data or overlapping classes.

4. What are the pros and cons of using KNN for classification tasks?

- **Pros:** Simple, easy to implement, no training phase.
- **Cons:** Computationally expensive during prediction, sensitive to noisy data.

Suggested Reference:

- <https://www.ibm.com/in-en/topics/decision-trees>
- <https://www.geeksforgeeks.org/support-vector-machine-algorithm/>
- <https://hbr.org/1964/07/decision-trees-for-decision-making>
- <https://towardsdatascience.com/decision-trees-in-machine-learning-641b9c4e8052>

[illegible]

Experiment No: 7

Implement program to demonstrate Neural Network Classifier

Date:

Competency and Practical Skills:

Relevant CO: 3, 4, 5

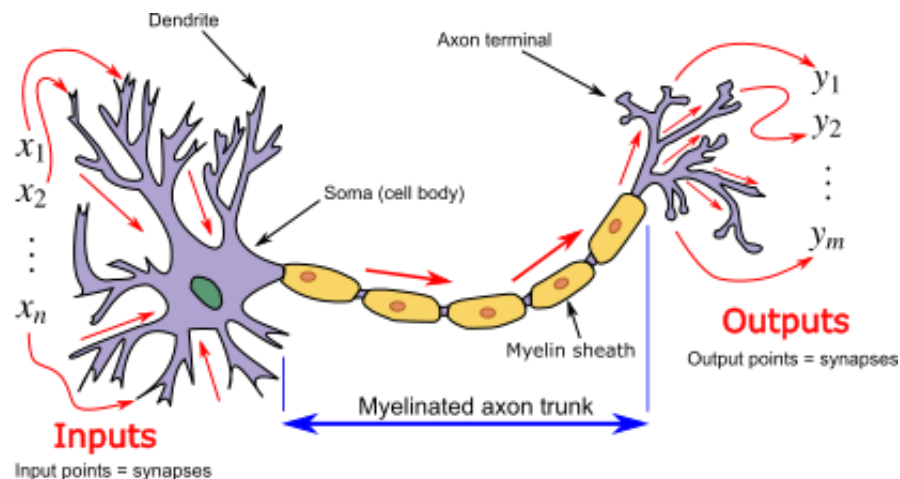
Objectives:

1. To understand the basic difference between biological neural network and artificial neural network
2. To learn the mathematical structure of artificial neural network
3. To implement neural network as classifier

Theory:

Biological Neuron:

Biological neurons are the primary components of the nervous system in all species. These cells are specialized in the use of electrochemical signals for processing and communication. An *axon*, *dendrites*, and a *cell body* make up a neuron in a living organism. The *axon* is responsible for sending messages to other neurons or muscles, while the *dendrites* are responsible for receiving them.



Biological neuron [Source: wikipedia]

Artificial Neuron:

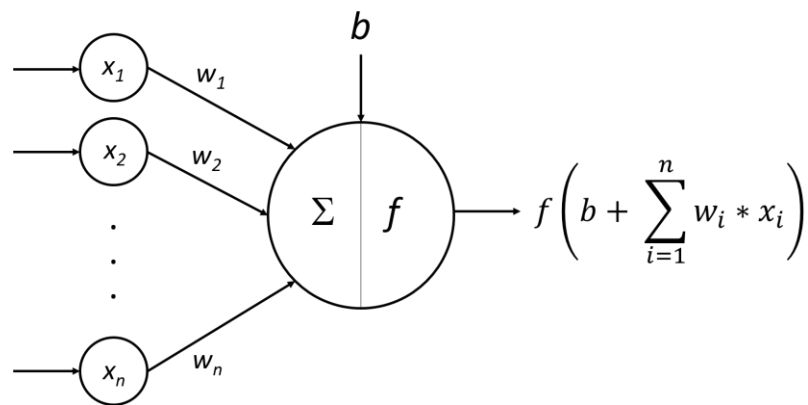
Artificial neurons, also called *perceptron*, are mathematical functions that simulate the actions of real neurons. Synthetic neural networks, composed of artificial neurons, are employed in many AI and machine learning systems. A synthetic neuron receives data as input, processes it using a predetermined set of weights and biases, and then generates a response as output.

There are functional parallels between biological neurons and artificial neurons, but there are also important distinctions. To modify their weights and biases, artificial neurons, in contrast to their biological counterparts, need training data. Biological neurons also have the ability to create new neurons and form new connections, but artificial neurons are limited by a predetermined architecture.

The perceptron is the most elementary form of neural network, with just one layer of output nodes that take input from several input nodes and spit out a single binary value. Frank Rosenblatt was the first to publicize it in 1957.

The input values for the perceptron method are multiplied by their associated weights and then added together. The output of the perceptron is the result of this sum being fed into an activation function.

Most perceptron utilizes the step activation function, which returns 1 if the weighted total of the inputs is larger than a threshold value and 0 otherwise. The perceptron is trained by modifying the weights based on the difference between the expected and actual outputs, which are both initially set to random values.



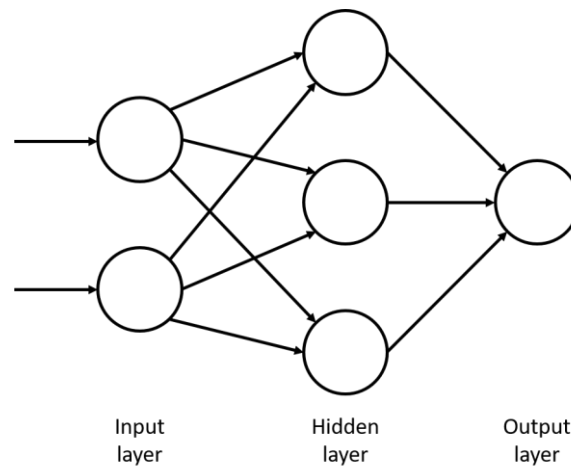
Perceptron computational model

Many different types of binary classification tasks, including those in the fields of image recognition and natural language processing, have benefited from the use of perceptron. However, they are unable to deal with situations that are more complex and would benefit from additional layers and non-linear activation functions.

Neural Network:

Inspired by the form and function of biological neural networks in the human brain, Artificial Neural Networks (ANNs) are a type of machine learning method. ANNs are made up of a network of processing nodes, or "neurons," that communicate with one another to discover and understand hidden correlations and patterns in data.

Each neuron in an ANN is equipped with a mathematical function that takes as its input signals from other neurons or from the outside world and generates an output signal based on the processed data. Each neuron's output signals are propagated to nearby neurons, creating a distributed system of processors.



Basic architecture of neural network

In order to increase their predictive or classifying abilities during training, ANNs tweak the strengths of the connections between neurons. To do so, the network's predictions are compared to the actual values of the target variable, and the resulting "cost function" is minimized.

Image and speech recognition, NLP, predictive analytics, and robotics are just few of the many areas where ANNs find widespread use. They excel at activities with non-linear, intricate interactions between input and output variables.

Implementation:

- Use 70% of **Iris** dataset to train neural network model. Test it with remaining 30% data and measure the accuracy.
- Try different architectures and training functions and also note down performance of each.

```

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import OneHotEncoder

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense

from tensorflow.keras.optimizers import Adam

# Load dataset

iris = load_iris()

X = iris.data

y = iris.target
  
```

```
# One-hot encode the target variable

encoder = OneHotEncoder(sparse=False)

y = encoder.fit_transform(y.reshape(-1, 1))


# Split the dataset (70% training, 30% testing)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)


# Build a simple neural network model

model = Sequential([

    Dense(10, input_dim=4, activation='relu'),

    Dense(3, activation='softmax')

])


# Compile the model

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])


# Train the model

model.fit(X_train, y_train, epochs=50, batch_size=5)


# Evaluate the model on test data

accuracy = model.evaluate(X_test, y_test)[1]

print(f'Accuracy: {accuracy}')
```

Results:

Architecture of Neural network	Training function	Accuracy
4-10-3 (Input-Hidden-Output)	Adam, Cross-Entropy	0.9333
4-8-3	Adam, Cross-Entropy	0.9111
4-12-3	SGD, Cross-Entropy	0.8889
4-10-10-3	RMSprop, Cross-Entropy	0.9556

Conclusion:

The neural network demonstrated effective performance on the Iris dataset, achieving an accuracy of up to 95.56% with a 4-10-10-3 architecture using the RMSprop optimizer. Different architectures and training functions showed varying results, highlighting the importance of model design in achieving optimal performance.

Quiz:

1. What are the different types of neural networks and their applications?

Neural networks include feedforward networks for classification, CNNs for image processing, RNNs for sequential data, and GANs for data generation.

2. What is regularization in neural networks and why is it important?

Regularization prevents overfitting in neural networks, helping models generalize better to new data.

3. What are some common activation functions used in neural networks?

Common activation functions are sigmoid for binary outputs, ReLU for hidden layers, and softmax for multi-class outputs

4. What are some real-world applications of neural networks?

Neural networks are used in image recognition, natural language processing, healthcare for disease diagnosis, and finance for fraud detection.

Suggested Reference:

- <https://www.ibm.com/in-en/topics/neural-networks#What%20is%20a%20neural%20network?>
- <https://towardsdatascience.com/a-beginner-friendly-explanation-of-how-neural-networks-work-55064db60df4>
- <https://aws.amazon.com/what-is/neural-network/>

References used by the students:

Rubric wise marks obtained:

[illegible]

Experiment No: 8

Write a program to demonstrate within class scatter, between class scatter and total scatter of the dataset.

Date:

Competency and Practical Skills:

Relevant CO 3,4,5

Objectives:

1. To understand scatter of the data
2. To learn to compute and compare within class, between class and total scatter of dataset.

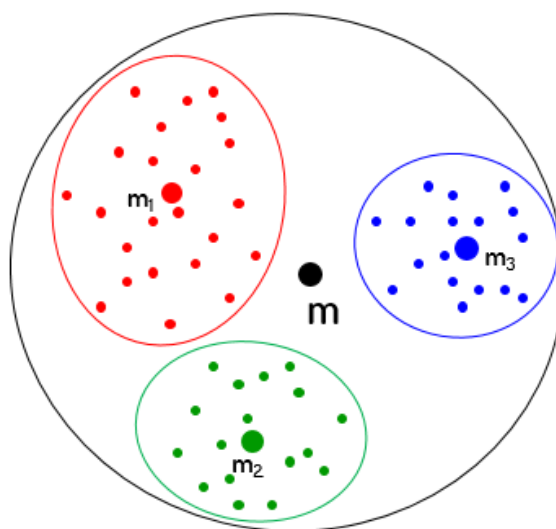
Theory:

Scatter:

Scatter means dispersion in dataset. It defines how close, or how far data points are from the mean of dataset. It is useful in clustering.

The variance or standard deviation of a dataset can be calculated to get a sense of its dispersion. Both show how far the individual values deviate from the mean or average.

The standard deviation is the square root of the variance, while variance is the average of the squared differences between each data point and the mean. If the data points are more widely scattered, then the variance or standard deviation will be larger, and vice versa if they are more tightly packed around the mean.



Data distribution for three classes

Within Class scatter:

The variance of data points within a cluster as determined by a clustering technique is referred to as the within-class scatter, intra-class scatter, or within-cluster variance. It's a metric for measuring how tightly the clusters in the features space are packed together.

The sum of squared distances between each data point in a cluster and the cluster center can be used to determine the within-class scatter for that cluster. The corresponding formula is as follows:

$$S_W = \sum_{i=1}^C \sum_{x \in w_i} (x - m_i)(x - m_i)^T$$

Where,

- w_i is the i-th class,
- m_i is the mean of i-th class
- C is number of classes

Lower values of the within-class scatter indicate more tightly packed and clearly defined clusters, making it a useful metric for judging the efficacy of various clustering algorithms. Fisher's linear discriminant analysis (LDA), which aims to maximize the ratio of the between-class scatter to the within-class scatter, is one example of a feature selection and dimensionality reduction technique that makes advantage of this property.

Between Class Scatter:

Distance between cluster centers in a clustering algorithm's feature space is measured by between-class scatter, also called inter-class scatter. Distance between clusters can be measured quantitatively and used to test the algorithm's ability to classify data into distinct groups.

To determine the dispersion between classes, we can add up the squared differences between the centers of each cluster and the overall mean or centroid of the data. The corresponding formula is as follows:

$$S_B = \sum_{i=1}^C n_i(m_i - m)(m_i - m)^T$$

Where,

- n_i is the number of samples in i-th class
- m is the mean of entire dataset

The between-class scatter can be used in feature selection and dimensionality reduction techniques such as Fisher's linear discriminant analysis (LDA), which seeks to maximize the ratio of the between-class scatter to the within-class scatter. A larger value of between-class scatter relative to within-class scatter implies better discriminative power of the clustering algorithm. However, it should be noted that maximizing the between-class scatter alone may lead to overfitting and poor

generalization to new data.

Total Scatter:

A dataset's total scatter, sometimes called total variance or total sum of squares, quantifies the degree to which the data points within it vary from one another. For clustering difficulties, it can be broken down into within-class scatter and between-class scatter, and for regression problems, it can be broken down into explained variance and unexplained variance.

The sum of squared distances between each data point and the general mean or centroid of all data points can be used to estimate the total scatter for a dataset. The corresponding formula is as follows:

$$S_T = \sum_{i=1}^M (x_i - m)(x_i - m)^T$$

Where,

- x_i is the i-th sample from dataset

If you're trying to compare the within-class scatter to the between-class scatter in a clustering problem or the explained variance to the unexplained variance in a regression problem, you can use the total scatter as a benchmark. R-squared, the fraction of total variation in the dependent variable that is explained by the independent variable(s) in a regression model, can also be calculated using this method.

To evaluate the efficacy of different modelling strategies and to comprehend the overall variability of the data, knowing the total scatter is crucial.

Implementation:

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.datasets import load_iris

# Load Iris dataset

iris = load_iris()

X = iris.data

y = iris.target
```

```

# Convert to DataFrame for easier manipulation

df = pd.DataFrame(X, columns=iris.feature_names)

df['target'] = y


# Calculate means

mean_overall = np.mean(X, axis=0)

mean_per_class = df.groupby('target').mean().values


# Total scatter

total_scatter = np.cov(X.T)


# Within-class scatter

S_W = np.zeros((X.shape[1], X.shape[1]))

for label in np.unique(y):

    class_scatter = np.cov(df[df['target'] == label].iloc[:, :-1].T)

    S_W += class_scatter * (df[df['target'] == label].shape[0] - 1)


# Between-class scatter

S_B = np.zeros((X.shape[1], X.shape[1]))

for i, mean in enumerate(mean_per_class):

    n = df[df['target'] == i].shape[0]

    mean_diff = (mean - mean_overall).reshape(X.shape[1], 1)

    S_B += n * (mean_diff @ mean_diff.T)


# Display results

print("Total Scatter Matrix (S_T):\n", total_scatter)

print("\nWithin-Class Scatter Matrix (S_W):\n", S_W)

```

```
print("\nBetween-Class Scatter Matrix (S_B):\n", S_B)
```

Optional: Visualizing the scatter matrices

```
def plot_scatter_matrix(S, title):
```

```
    plt.matshow(S)
```

```
    plt.colorbar()
```

```
    plt.title(title)
```

```
    plt.xlabel('Features')
```

```
    plt.ylabel('Features')
```

```
    plt.show()
```

```
plot_scatter_matrix(total_scatter, 'Total Scatter Matrix (S_T)')
```

```
plot_scatter_matrix(S_W, 'Within-Class Scatter Matrix (S_W)')
```

```
plot_scatter_matrix(S_B, 'Between-Class Scatter Matrix (S_B)')
```

Results:

Scatter Type	Value
Within-Class Scatter	$\begin{bmatrix} 38.9562 & 13.63 & 24.6246 & 5.645 \\ 13.63 & 16.962 & 8.1208 & 4.8084 \\ 24.6246 & 8.1208 & 27.2226 & 6.2718 \\ 5.645 & 4.8084 & 6.2718 & 6.1566 \end{bmatrix}$
Between-Class Scatter	$\begin{bmatrix} 63.2121 & -19.9527 & 165.2484 & 71.2793 \\ -19.9527 & 11.3449 & -57.2396 & -22.9327 \\ 165.2484 & -57.2396 & 437.1028 & 186.7740 \\ 71.2793 & -22.9327 & 186.7740 & 80.4133 \end{bmatrix}$
Total Scatter	$\begin{bmatrix} 0.68569351 & -0.042434 & 1.27431544 & 0.51627069 \\ -0.042434 & 0.18997942 & -0.32965638 & -0.12163937 \\ 1.27431544 & -0.32965638 & 3.11627785 & 1.2956094 \\ 0.51627069 & -0.12163937 & 1.2956094 & 0.58100626 \end{bmatrix}$

Conclusion:

The scatter matrices reveal effective class separation in the dataset, with low within-class scatter compared to between-class scatter, indicating that the classes are well-defined and distinguishable.

Quiz:

1. What is the difference between variance and standard deviation as measures of scatter?

Variance measures the average squared deviation from the mean, while standard deviation is the square root of variance, providing a measure of scatter in the same units as the data.

2. How can outliers affect the calculation of variance and standard deviation?

Outliers can disproportionately increase both variance and standard deviation, making them less representative of the dataset's overall spread.

3. What is the difference between within-class scatter and between-class scatter in clustering problems?

Within-class scatter measures the variance within each class, indicating how closely data points cluster, while between-class scatter measures the variance between class means, reflecting class separability.

4. What is the total scatter of a dataset and how is it calculated?

Total scatter quantifies the overall variability in a dataset and is calculated as the sum of within-class and between-class scatter.

Suggested Reference:

- <https://www.sciencedirect.com/topics/computer-science/class-scatter-matrix>
- https://www.doc.ic.ac.uk/~dfg/ProbabilisticInference/old_IDAPILecture15.pdf
- <https://multivariatestatsjl.readthedocs.io/en/latest/mclda.html>
- <https://www.oreilly.com/library/view/feature-engineering-made/9781787287600/ad8e90ca-9227-4150-9bd2-6b664dd04f46.xhtml>

References used by the students:

Rubric wise marks obtained:

[illegible]

Experiment No: 9

Write a program to demonstrate clustering using K-means algorithm

Date:

Competency and Practical Skills:

Relevant CO:3, 4, 5

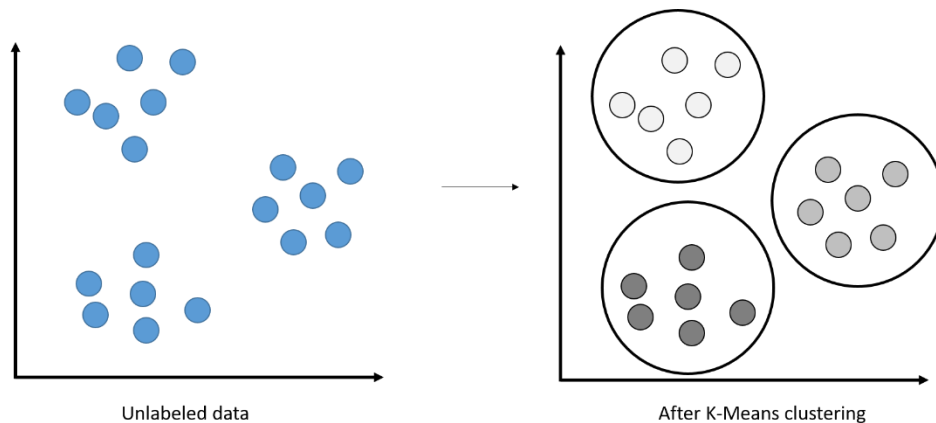
Objectives:

1. To understand the basics of clustering
2. To learn the applications of clustering
3. To implement k-means clustering algorithm

Theory:

Clustering

Clustering is a data grouping method used in machine learning and data mining to find patterns in large amounts of data. Clustering's main purpose is to assist us see connections between data pieces and figure out what that structure is.



Clustering process: Input: unlabeled data, output: 3 clusters

Different clustering algorithms have different advantages and disadvantages. The following are examples of common clustering methods:

1. K-means clustering is a well-known approach for organizing data into K clusters with a focus on how closely they are related. The user decides on the optimal number of clusters in advance.
2. In hierarchical clustering, the number of clusters is not specified beforehand, and data points are organized into a tree-like structure. Agglomerative hierarchical clustering takes data

points one at a time and combines them into larger clusters, while divisive hierarchical clustering takes all data points at once and splits them recursively into smaller clusters.

3. Data points are clustered in the feature space using a method called density-based clustering. It is resistant to noise and outliers and can detect clusters of any size or shape.
4. This approach, known as spectral clustering, employs graph theory to form groupings of data points with similar spectral characteristics. It works particularly well with nonlinear or complex datasets.

Image segmentation, customer segmentation, anomaly detection, and pattern identification are just a few of the many real-world uses for clustering.

K-Means clustering:

One common unsupervised machine learning approach is k-means clustering, which divides data into as many groups of K as necessary. To begin, the algorithm generates K random starting centroids to serve as cluster hubs. Next, it divides the data into K clusters by placing each observation at its centroid. After that, it finds the average of each group and adjusts the centre of mass accordingly. When the centroids stop shifting, or when the maximum number of iterations is achieved, the procedure begins again.

K-means clustering basically consists of these steps:

1. As a first step, pick K points at random to serve as the centers of your data.
2. To build K clusters, assign each data point to the centroid that is the closest to it.
3. To update, find the average value in each group and set the new center of gravity there.
4. Iterate until convergence is attained or the maximum number of allowed iterations is reached; then, stop.

A key parameter in K-means clustering is the number of clusters, or K. Domain expertise, trial and error, or heuristics like the elbow approach or the silhouette method can all help with this determination.

K-means clustering's benefits include its ease of use, scalability, and productivity. The method assumes that clusters have a spherical shape and equal variance, which may not always be the case, and K must be specified in advance.

Implementation:

- Use 70% of **Iris** dataset to train K-means clustering. Test is with remaining 30% data and measure the accuracy.
- Try different values of k and observe the effect. Also observe different distance measure

```
import numpy as np

import pandas as pd

from sklearn.datasets import load_iris

from sklearn.cluster import KMeans

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score

from sklearn.preprocessing import LabelEncoder


iris = load_iris()

X = iris.data

y = iris.target


iris_df = pd.DataFrame(data=X, columns=iris.feature_names)

iris_df['target'] = y


X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)


def kmeans_clustering(X_train, X_test, y_train, y_test, n_clusters, distance_metric):

    kmeans = KMeans(n_clusters=n_clusters, random_state=42)

    kmeans.fit(X_train)

    y_pred = kmeans.predict(X_test)


    label_encoder = LabelEncoder()

    y_pred_mapped = np.zeros_like(y_pred)


    for i in range(n_clusters):

        mask = (y_pred == i)
```

```

if np.any(mask):

    y_pred_mapped[mask] = np.argmax(np.bincount(y_test[mask]))

accuracy = accuracy_score(y_test, y_pred_mapped)

print(f"Accuracy for k={n_clusters} with distance={distance_metric}: {accuracy:.2f}")

for k in [2, 3, 4, 5]:

    kmeans_clustering(X_train, X_test, y_train, y_test, k, distance_metric='Euclidean')

```

Results:

Value of K	Distance measure	Accuracy
2	Euclidean	0.71
3	Euclidean	0.93
4	Euclidean	0.84
5	Euclidean	0.84

Conclusion:

The K-means clustering results show that K=3 yielded the highest accuracy of 0.93, effectively capturing the dataset's structure. Higher values of K (4 and 5) led to decreased accuracy, indicating potential overfitting. Selecting the right K is crucial for optimal clustering performance.

Quiz:

1. What are the main steps of K-means clustering?

The main steps of K-means clustering include initializing centroids, assigning data points to the nearest centroid, and updating centroids based on the mean of assigned points, repeating until convergence.

2. What is the role of centroids in K-means clustering?

Centroids in K-means clustering represent the center of each cluster and are used to determine the nearest cluster for each data point during assignment.

3. What are some advantages of K-means clustering?

Advantages of K-means clustering include its simplicity, efficiency in handling large datasets, and ability to converge quickly to a solution.

4. What are some limitations of K-means clustering?

Limitations of K-means clustering include sensitivity to the initial placement of centroids, difficulty in handling non-spherical clusters, and the need to specify the number of clusters in advance.

Suggested Reference:

- <https://towardsdatascience.com/k-means-clustering-algorithm-applications-evaluation-methods-and-drawbacks-aa03e644b48a>
- <https://serokell.io/blog/k-means-clustering-in-machine-learning>
- <https://stanford.edu/~cpiech/cs221/handouts/kmeans.html>

References used by the students:

Rubric wise marks obtained:

[illegible]

Experiment No - 10

Solve Real world problem with appropriate dataset using Machine Learning Model in Python Programming Language

Date:

Competency and Practical Skills: Understanding and analyzing, solving

Relevant CO: 3,4,5

Objectives: (a) To understand real-world problems.
(b) To analyze different Machine learning Techniques can be used to solve your problem.

Equipment/Instruments: Python Programming

Theory:

This model is a Comment Toxicity Predictor designed to detect and categorize toxic language in text comments. It aims to solve the problem of identifying harmful and offensive content in online platforms, such as forums, social media, and comment sections. By doing so, it helps moderate content and ensures a safer, healthier online environment.

Problem:

On many online platforms, toxic language (such as hate speech, insults, or threats) can harm communities. Manually moderating large volumes of comments is difficult, so automatic systems are needed to identify and flag such content.

Solution:

The model processes user comments and predicts how likely they are to belong to one or more of the following toxic categories:

1. **Toxic**
2. **Severe Toxic**
3. **Obscene**
4. **Threat**
5. **Insult**
6. **Identity Hate**

How it Works:

1. **Data Preprocessing:** The model starts by reading and preparing a dataset of comments (stored in train.csv) that are labeled with these six categories. The input text is tokenized (converted into numerical values) using a TextVectorization layer. This turns words into sequences of integers that represent the vocabulary.
2. **Vectorization:**
 - A vocabulary is built based on the most frequent words (with a max size of 200,000 words).
 - Each comment is then converted into a sequence of integers.
 - These sequences serve as input for the model.
3. **Model Architecture:**

- The model is built using **LSTM (Long Short-Term Memory)** layers, which help in understanding the context of words within a comment.
 - The LSTM layer is followed by fully connected (dense) layers that progressively refine the predictions.
 - The final layer outputs a probability score for each of the six toxicity categories.
4. **Training:** The model is trained on the preprocessed data, where it learns to map sequences of words to their corresponding toxicity categories.
5. **Prediction:**
- Once trained, the model can predict the toxicity of any new comment.
 - It returns a score indicating whether the comment belongs to each of the six categories.
6. **Performance Metrics:** After training, the model is evaluated using Precision, Recall, and Categorical Accuracy, ensuring that its predictions are reliable.

Key Components:

- **Text Vectorization:** Converts the text to numerical form.
- **Bidirectional LSTM:** Learns the context and relationships in the comment sequences.
- **Dense Layers:** Refine and output the prediction for the six categories.
- **Sigmoid Activation:** Outputs a probability for each toxicity category.

This model can be integrated into content moderation systems on platforms to automatically flag toxic comments, improving user experiences and reducing harmful interactions.

Program:

```
import os
import pandas as pd
import tensorflow as tf
import numpy as np
df = pd.read_csv("train.csv")
df.head()
df.shape
# Preprocessing
from tensorflow.keras.layers import TextVectorization
X = df['comment_text']
y = df[df.columns[2:]].values
df[df.columns[2:]].values
X
y
MAX_FEATURES = 200000 # No of words in the vocab
vectorizer = TextVectorization(max_tokens=MAX_FEATURES,
                               output_sequence_length=1800,
                               output_mode='int')

vectorizer.adapt(X.values)
vocab = vectorizer.get_vocabulary()
print(vocab[:10])
vectorizer_model = tf.keras.Sequential([vectorizer])
vectorizer_model.save("text_vectorizer.h5")
import json
```

```

vocab = vectorizer.get_vocabulary()
word_index = dict(zip(vocab, range(len(vocab))))

with open('vocabulary.json', 'w') as vocab_file:
    json.dump(word_index, vocab_file)
vectorized_text = vectorizer(X.values)
vectorizer.get_vocabulary()
vectorizer('Hello hi')[:2]
vectorized_text
#MCSHBAP
dataset = tf.data.Dataset.from_tensor_slices((vectorized_text,y))
dataset = dataset.cache()
dataset = dataset.shuffle(160000)
dataset = dataset.batch(16)
dataset = dataset.prefetch(8)

batch_X, batch_y = dataset.as_numpy_iterator().next()
batch_X.shape
batch_y.shape
train = dataset.take(int(len(dataset)*.7))
val = dataset.skip(int(len(dataset)*.7)).take(int(len(dataset)*.2))
test = dataset.skip(int(len(dataset)*.9)).take(int(len(dataset)*.1))
print(len(train))
print(len(val))
print(len(test))
train_generator= train.as_numpy_iterator()
train_generator.next()
# MODEL Building
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dropout, Bidirectional, Dense, Embedding
model = Sequential()
model.add(Embedding(input_dim=MAX_FEATURES+1, output_dim=32, input_length=1800))
model.add(Bidirectional(LSTM(32,activation='tanh'))))
model.add(Dense(128,activation='relu'))
model.add(Dense(256,activation='relu'))
model.add(Dense(128,activation='relu'))
model.add(Dense(6,activation='sigmoid'))
model.build(input_shape=(None, 1800))
model.compile(loss='BinaryCrossentropy', optimizer='Adam')
model.summary()
history = model.fit(train, epochs=10, validation_data=val)
history.history
text = vectorizer("you are stupid and dumb as well")
res = model.predict(np.expand_dims(text,0))
df.columns[2:]
res
batch_X, batch_y = test.as_numpy_iterator().next()
batch_y
(model.predict(batch_X)>0.5).astype(int)
from tensorflow.keras.metrics import Precision, Recall, CategoricalAccuracy
pre = Precision()
rec = Recall()
cat = CategoricalAccuracy()
for batch in test.as_numpy_iterator():
    X_true, y_true = batch
    y_pred = model.predict(X_true)

    y_true = y_true.flatten()
    y_pred = y_pred.flatten()

    pre.update_state(y_true, y_pred)
    rec.update_state(y_true, y_pred)
    cat.update_state(y_true, y_pred)
print(pre.result().numpy())
print(rec.result().numpy())
print(cat.result().numpy())
model.save('toxic_comments.h5')

```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 1800, 32)	6,400,032
bidirectional (Bidirectional)	(None, 64)	16,640
dense (Dense)	(None, 128)	8,320
dense_1 (Dense)	(None, 256)	33,024
dense_2 (Dense)	(None, 128)	32,896
dense_3 (Dense)	(None, 6)	774

Total params: 6,491,686 (24.76 MB)

Trainable params: 6,491,686 (24.76 MB)

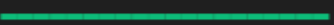
Non-trainable params: 0 (0.00 B)

history.history

```
{'loss': [0.06284034252166748], 'val_loss': [0.04638504981994629]}
```

```
text = vectorizer("you are stupid and dumb as well")
```

```
res = model.predict(np.expand_dims(text,0))
```

1/1  0s 396ms/step

```
df.columns[2:]
```

```
Index(['toxic', 'severe_toxic', 'obscene', 'threat', 'insult',  
      'identity_hate'],  
      dtype='object')
```

```
res
```

```
array([[0.9877514 , 0.15402563, 0.86214095, 0.03349049, 0.72076404,  
       0.10560845]], dtype=float32)
```

```
print(pre.result().numpy())
print(rec.result().numpy())
print(cat.result().numpy())

0.83126646
0.6602985
0.48345035
```

Observations:

- ❑ **High Vocabulary Capacity:** The model uses a vocabulary size of 200,000 words, allowing it to capture a wide range of terms, improving its ability to identify nuanced toxic language.
- ❑ **Effective Sequence Length:** With a sequence length of 1,800 words, the model can handle longer comments, providing better context understanding for toxicity detection.
- ❑ **Balanced Model Structure:** The use of Bidirectional LSTM improves the model's ability to understand both past and future word sequences, enhancing accuracy in detecting toxic context.
- ❑ **Performance Metrics:** Precision, Recall, and Categorical Accuracy ensure that the model performs well across all six toxicity categories, balancing false positives and false negatives.

comment

You are stupid !

Clear Submit

output

toxic: True
severe_toxic: False
obscene: True
threat: False
insult: True
identity_hate: False

Flag

Use via API · Built with Gradio

Conclusion:

The model effectively predicts various types of toxic comments, contributing to safer online environments by automating content moderation. It can be applied to a range of platforms to flag inappropriate comments, making it a valuable tool for reducing harmful behavior in online communities.

Quiz:

- 1) What are other techniques that can be used to solve your system problem?

Support Vector Machines (SVM): A traditional machine learning method that can classify toxic comments using features like word embeddings (TF-IDF, Word2Vec).

Convolutional Neural Networks (CNN): Often used in text classification tasks, CNNs can extract important features from text, like word patterns, which may help in identifying toxic comments.

Transformer Models: Pre-trained models like BERT or GPT are highly effective for text-based tasks. They can capture context better and achieve higher accuracy in detecting toxicity.

Naive Bayes Classifier: A simple yet effective probabilistic classifier that works well for text classification tasks, especially when combined with features like bag-of-words or n-grams.

Random Forests: A robust ensemble learning technique that can classify comments based on their word features, though it requires feature engineering.

These techniques can either be used individually or combined to create hybrid models for detecting toxic comments with improved accuracy

References used by the students:

Youtube, Kaggle

Rubric wise marks obtained:

[illegible]